

Complements of Operational Research

3.2 Sequencing problems: three or more machines

José Firmino Aguilar Madeira

The print shop buys a guillotine

Between printing and finishing comes a third machine, **cutting** (guillotine). The six jobs of 3.1 now go through **three** machines, always in the same order.

	Job	Printing	Cutting	Finishing
A	Catalogues	8	2	9
B	Magazines	11	3	5
C	Cards	5	1	2
D	Invitations	3	2	5
E	Books	4	3	12
F	Posters	10	1	9

times in hours

With three machines there is no longer a simple rule that always gives the optimum (the problem is **NP-hard** for $m \geq 3$).

But: if the middle machine is “fast” compared with the others, it is never the one holding up the line, and the problem behaves like a **two**-machine problem.

This is the case with cutting: at most 3 h, against at least 3 h of printing and 2 h of finishing.

Johnson with three machines

Condition (Johnson, 1954)

Johnson's rule can be applied if **at least one** of the conditions holds:

$$\min_j M_{1j} \geq \max_j M_{2j} \quad \text{or} \quad \min_j M_{3j} \geq \max_j M_{2j}.$$

In that case, two **fictitious** machines $G_j = M_{1j} + M_{2j}$ and $H_j = M_{2j} + M_{3j}$ are created and Johnson is applied to (G, H) . The sequence obtained is optimal for the three machines.

In the example: $\min M_1 = 3$ (D)

$\geq \max M_2 = 3$ (B, E). ✓

(The other one fails: $\min M_3 = 2 < 3$. One is enough.)

	A	B	C	D	E	F
$G = M_1 + M_2$	10	14	6	5	7	11
$H = M_2 + M_3$	11	8	3	7	15	10

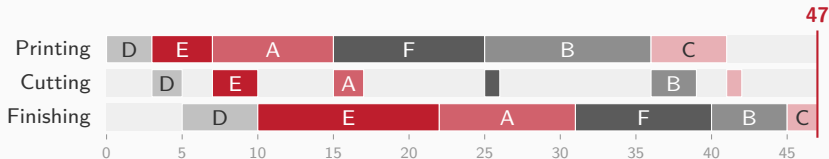
Result: D, E, A, F, B, C with $T = 47$ h

Johnson on (G, H) :

- $G \leq H$: D (5), E (7), A (10) by increasing G ;
- $G > H$: F (10), B (8), C (3) by decreasing H .

Completion	D	E	A	F	B	C
Printing	3	7	15	25	36	41
Cutting	5	10	17	26	39	42
Finishing	10	22	31	40	45	47

The same sequence as in 3.1 (cutting is so fast that it hardly counts).



Idle: printing 6 h, cutting 35 h, finishing 5 h. In order of arrival: 55 h. **Brute force:** the minimum of the 720 sequences is 47 h — confirmed.

Condition and reduction

If $\min_j M_{1j} \geq \max_j M_{ij}$ for $i = 2, \dots, m-1$, or $\min_j M_{mj} \geq \max_j M_{ij}$ for $i = 2, \dots, m-1$, then Johnson is applied to the fictitious machines

$$G_j = M_{1j} + M_{2j} + \dots + M_{m-1,j}, \quad H_j = M_{2j} + M_{3j} + \dots + M_{mj}.$$

Idea: the middle machines are never the bottleneck. The first or the last one “rules” the line.

With $m = 3$ this is the condition of the previous slide.

In the tests: you must always **check the condition first** (“if possible, solve with Johnson’s method”). If it fails, the rule may give a sequence that is not optimal — next slide.

When the condition fails: the laminating week

In another week, four jobs need **laminating** — slow — between printing and finishing.

Job	Print.	Lamin.	Finish.
1	2	4	3
2	7	8	8
3	2	9	7
4	4	8	5

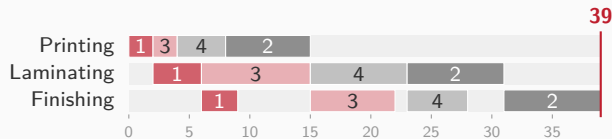
$\max M_2 = 9$, but $\min M_1 = 2$ and $\min M_3 = 3$:
the condition fails.

Applying Johnson anyway to

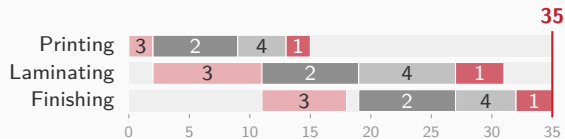
$G = (6, 15, 11, 12)$, $H = (7, 16, 16, 13)$:

sequence 1, 3, 4, 2.

Johnson “by force”: 1, 3, 4, 2 $\Rightarrow T = 39$



Optimum (brute force, 24 sequences): 3, 2, 4, 1 $\Rightarrow T = 35$



Laminating is the bottleneck: the optimum keeps it working non-stop from early on (it starts with job 3, which has a short printing and a long laminating).

What now?

Without the condition, Johnson becomes just a **heuristic**: it gives a reasonable sequence, with no guarantee.

How bad can it be?

In 500 random problems with 3 machines and 7 jobs (times from 1 to 20) **without** the condition: Johnson hit the optimum in 49% of the cases; on average it was 3% above; in the worst case, 21% above.

Two ways out, which come next:

1. **Another heuristic** — the TDM (*time deviation method*), in 3.3. Fast; also without a guarantee.
2. **An exact method** that does not enumerate everything — *branch and bound*, in 3.4. Guarantees the optimum (in the example: 35 h).

To solve in class

Example 3 — three machines

Job	1	2	3	4	5
M1	8	10	6	7	11
M2	5	6	2	3	4
M3	4	9	8	6	5

Example 4 — three machines

Job	1	2	3	4	5
M1	6	7	9	10	6
M2	5	6	4	3	4
M3	11	10	6	7	8

Example 5 — five machines

Job	1	2	3	4
M1	7	6	5	8
M2	5	6	4	3
M3	2	4	5	3
M4	3	5	6	2
M5	9	10	8	6

In each case: check the condition; determine the sequence, the total time and the idle times of each machine.

KEY TAKEAWAYS — 3.2

1. With $m \geq 3$ machines, **first check the condition**: the minimum of the first (or of the last) machine $\geq$ the maximum of each intermediate machine.
2. If it holds: $G = M_1 + \dots + M_{m-1}$, $H = M_2 + \dots + M_m$, and Johnson on (G, H) gives the **optimum**.
3. Compute the total time with the m real machines (not with G and H).
4. If the condition fails, Johnson is just a heuristic (in the example: 39 h against the optimum of 35 h).

Next: 3.3 The time deviation method (TDM) — a heuristic

References and resources

References Full list: *Course bibliography*.

- Johnson, S. M., Optimal two- and three-stage production schedules with setup times included, *Naval Research Logistics Quarterly*, 1(1), 61–68, 1954.
- Kasana, H. S., Kumar, K. D., *Introductory Operations Research*, Springer, 2004 — ch. 10 (n jobs, m machines).
- Garey, M. R., Johnson, D. S., Sethi, R., The complexity of flowshop and jobshop scheduling, *Mathematics of Operations Research*, 1(2), 117–129, 1976.
- Pinedo, M. L., *Scheduling: Theory, Algorithms, and Systems*, 5th ed., Springer, 2016.

Resources

- Exercise sheet 3. `code/python/03_scheduling/03_2_m_machines/m_maquinas.py`.